TRAVAUX DIRIGÉS N° 1

Objectifs pédagogiques : manipuler les variables et les expressions, construire des conditions, utiliser les boucles, valider les saisies et traduire un algorithme simple en C++.

Consignes générales

- Pour chaque exercice, identifier les données d'entrée, les traitements et les résultats attendus.
- Écrire d'abord l'algorithme en pseudocode, sauf indication contraire.
- Tester les solutions avec les cas proposés et ajouter au moins un cas limite ou invalide.
- Soigner l'indentation et donner des noms explicites aux variables.

Exercice 1 — Calculer une facture avec remise

Objectif : Lecture/écriture, calculs, conditions et validation des données.

Une boutique vend des clés USB et des souris. Le programme lit le prix unitaire et la quantité de chaque article. Il calcule le montant brut. Si le montant brut est supérieur ou égal à 20 000 MRU, une remise de 5 % est accordée. Les quantités doivent être positives ou nulles et les prix strictement positifs.

1. Préciser les entrées, les traitements et les sorties.
2. Écrire l'algorithme qui calcule le montant brut, la remise et le montant net.
3. Prévoir un message d'erreur si un prix ou une quantité ne respecte pas les règles.
4. Tester avec : clé USB à 500 MRU (20 unités) et souris à 1 500 MRU (8 unités).
5. Tester également une commande dont le montant brut est inférieur à 20 000 MRU.

Exercice 2— Contrôle d'accès à une plateforme universitaire

Objectif : Expressions booléennes, opérateurs logiques et conditions imbriquées.

Une plateforme universitaire autorise la connexion si l'identifiant est valide, si le mot de passe est valide et si le compte n'est pas bloqué. On dispose des variables booléennes identifiantValide, motDePasseValide et compteBloque.

1. Écrire la condition logique qui autorise la connexion.
2. Écrire un algorithme qui affiche « Accès autorisé » ou un message d'erreur adapté.
3. Tester les cas suivants : toutes les données valides ; identifiant invalide ; mot de passe invalide ; compte bloqué.
4. Réécrire la condition en utilisant explicitement NON et ET.

Exercice 3 — Calculer une facture d'électricité progressive

Objectif : Conditions simples et multiples, opérateurs arithmétiques et validation.

Une facture simplifiée est calculée ainsi : les 100 premiers kWh coûtent 5 MRU/kWh ; les 100 kWh suivants coûtent 8 MRU/kWh ; toute consommation au-delà de 200 kWh coûte 12

MRU/kWh. Le tarif est progressif : chaque tranche est facturée à son propre tarif. Une consommation négative est invalide.

1. Écrire l'algorithme qui lit la consommation et calcule le montant total.
2. Traiter les cas : consommation ≤ 100 ; comprise entre 100 et 200 ; supérieure à 200.
3. Calculer à la main puis vérifier les résultats pour 80 kWh, 150 kWh et 250 kWh.
4. Expliquer la différence entre un tarif progressif et un tarif unique appliqué à toute la consommation.

Pour aller plus loin : Ajouter une taxe de 2 % sur le montant calculé.

Exercice 4 — Calculatrice à menu

Objectif : Boucle TANT QUE, choix conditionnels, opérateurs et traitement des erreurs.

Réaliser une calculatrice qui propose le menu : 1. Addition ; 2. Soustraction ; 3. Multiplication ; 4. Division ; 0. Quitter.

L'utilisateur choisit une opération, saisit deux nombres et obtient le résultat. Le menu réapparaît jusqu'au choix 0.

1. Écrire l'algorithme en utilisant une boucle TANT QUE.
2. Afficher un message si le choix du menu est invalide.
3. Empêcher la division par zéro.
4. Faire une trace avec une addition, une division par zéro, un choix invalide et le choix 0.

Exercice 5 — Bilan des résultats d'un groupe

Objectif : Boucle POUR, compteurs, accumulateurs, maximum et pourcentage.

Un enseignant veut analyser les notes de N étudiants. Le programme lit N, puis les N notes comprises entre 0 et 20. Il calcule le nombre d'admis (note ≥ 10), le nombre de non-admis, la moyenne, la meilleure note et le taux de réussite. Toute note hors de [0, 20] doit être redemandée.

1. Écrire l'algorithme avec une boucle POUR.
2. Initialiser correctement les compteurs et l'accumulateur de somme.
3. Calculer le taux de réussite en pourcentage.
4. Tester avec N = 5 et les notes 12, 8, 15, 9 et 16.
5. Que faut-il faire si N est nul ou négatif ?

Exercice 6 — Saisie sécurisée d'un code PIN

Objectif : Validation de saisie et comparaison des boucles WHILE et DO-WHILE.

Un programme demande un code PIN composé de quatre chiffres, représenté ici par un entier compris entre 1000 et 9999. Tant que la valeur est invalide, le programme affiche un message et redemande la saisie.

1. Écrire l'algorithme avec DO-WHILE.
2. Écrire une deuxième version avec WHILE.
3. Expliquer la différence de fonctionnement entre ces deux boucles.

4. Tester avec 25, 12345 et 4821.

Exercice 7 — Jeu de devinette avec nombre d'essais limité

Objectif : Boucles, conditions imbriquées, compteur et traduction en C++.

Le nombre secret est 42. L'utilisateur dispose de 3 essais pour le deviner. Après chaque proposition incorrecte, le programme indique si le nombre secret est plus grand ou plus petit. Le jeu s'arrête lorsque le nombre est trouvé ou lorsque les 3 essais sont épuisés.

1. Écrire un programme n C++.
2. Afficher le nombre d'essais utilisés en cas de réussite.
3. Afficher un message de fin et révéler le nombre secret en cas d'échec.
4. Tester une réussite au premier essai, une réussite au septième essai et un échec.
5. Vérifier que le jeu ne demande pas un huitième essai.

Notion	Exercice(s)
Variables, types, affectation, trace	Exercice 1
Entrées, sorties, calculs et validation	Exercice 2
Booléens, NON, ET, conditions imbriquées	Exercice 3
Conditions simples et multiples	Exercice 4
TANT QUE, menu, erreurs de calcul	Exercice 5
POUR, compteurs, somme, moyenne, maximum	Exercice 6
RÉPÉTER…JUSQU'À et TANT QUE	Exercice 7
Compteur, conditions et traduction en C++	Exercice 8

Conseil de méthode. Avant de coder, effectuer une trace manuelle de l'algorithme et vérifier les cas ordinaires, les cas limites et les saisies invalides.